

Danmarks
Tekniske
Universitet



Software Engineering Project Report 2

02161 Software Engineering I
DTU Compute, Spring 2026

Group 16

AUTHORS

Nicolai Søgaard -s234146
Nat Skovlund - s245203
Mathias Kjølbye - s240803
Vedanta Sinha - s245010
Viktor Munk - 240335

Git Repository:

<https://gitlab.gbar.dtu.dk/XXXX/software-project>
30. april 2026

Indhold

1	Introduction	1
1.1	Scope	1
1.2	Relation to Report 1	1
2	Implemented System Overview	2
2.1	Implemented Functionality	2
2.2	Architecture Overview	2
2.3	Non-Implemented Features	2
2.4	Traceability to Report 1	2
3	Systematic Tests (White-Box Tests)	3
3.1	Choice of Methods	3
3.2	<code>ProjectService.createProject(...)</code>	3
3.2.1	Method	3
3.2.2	Input Properties	3
3.2.3	Concrete Test Cases	3
3.2.4	Implementation of Tests	4
3.3	<code>ActivityService.createActivity(...)</code>	4
3.3.1	Method	4
3.3.2	Input Properties	4
3.3.3	Concrete Test Cases	4
3.4	<code>ActivityService.addDeveloperToActivity(...)</code>	4
3.4.1	Method	4
3.4.2	Input Properties	5
3.4.3	Concrete Test Cases	5
3.5	<code>TimeRegistrationService.registerTime(...)</code>	5
3.5.1	Method	5
3.5.2	Input Properties	5
3.5.3	Concrete Test Cases	6
3.6	<code>AbsenceService.registerAbsence(...)</code>	6
3.6.1	Method	6
3.6.2	Input Properties	6
3.6.3	Concrete Test Cases	6
3.7	Summary of White-Box Testing	6
4	Design by Contract (Vedanta Sinha)	8
4.1	<code>ProjectService.createProject(...)</code>	8
4.1.1	Method with Assertions	8
4.1.2	Contract	8
4.1.3	Discussion	8
4.2	<code>ActivityService.createActivity(...)</code>	8
4.2.1	Method with Assertions	8

4.2.2	Contract	9
4.3	<code>ActivityService.addDeveloperToActivity(...)</code>	9
4.3.1	Method with Assertions	9
4.3.2	Contract	9
4.4	<code>TimeRegistrationService.registerTime(...)</code>	10
4.4.1	Method with Assertions	10
4.4.2	Contract	10
4.5	<code>AbsenceService.registerAbsence(...)</code>	11
4.5.1	Method with Assertions	11
4.5.2	Contract	11
4.6	Discussion of Contracts	12
5	Code Coverage (Viktor)	13
5.1	Measurement Setup	13
5.2	Results	13
5.3	Analysis by Layer	14
5.3.1	Repository layer 89.16%	14
5.3.2	UI layer 0%	14
5.3.3	Service layer 77.25%	14
5.3.4	Domain layer 79.77%	15
5.4	Effect of the White-Box Tests	16
5.5	Limitations	17
5.6	Summary	17
6	Design Patterns and SOLID Principles (nicolai)	18
6.1	Overall Design Choices	18
6.1.1	Domain Layer	19
6.1.2	Repository Layer	19
6.1.3	Service layer	20
6.1.4	User Interface Layer	21
6.2	SOLID-principles (dobbeltkonfekt skal rettes ti li final)	21
6.3	Single Responsible Principle	22
6.3.1	Domain-classes	22
6.3.2	Repositories	22
6.3.3	Service layer	23
6.4	Dependancy Inversion Principle	23
6.5	Other SOLID-Principles	25
6.5.1	OCP, Open/Close principle	25
6.5.2	Interface Segregation Principle	25
6.5.3	Liskov Substitution Principle	26
7	Conclusion	27
7.1	Reflection on the Development Process	27
7.2	Collaboration and Contribution Weights	27

8 Contribution Table	28
9 Appendix	29
9.1 Git Repository	29
9.2 Run Instructions	29
List of Figures	30
List of Tables	31
References	32

1 Introduction

This report shows the implementation and testing of the project management system, which allows admins to create projects and activities for the former, allocate developers to projects, use a project's allocated time to track progress, and generate reports to keep teams, management, and stakeholders aligned.

The main objectives of this report are to systematically conduct white box testing on the different methods in the code. Covering the code and results after executing all the necessary testing. All the while using design patterns and SOLID principles. Finally, an evaluation of the implemented system compared to the original design seen in Report 1.

For this project, we also aim to maintain traceability between requirements, design, implementation, and tests.

1.1 Scope

The main emphasis of this project is placed on the core service layer functionality. This includes the prior mentioned project and activity management, resource allocation, progress tracking, and reporting. This is the most relevant part for detailed analysis, since it contains the central application logic. We also worked on creating a clean and professional graphical user interface (GUI) to realistically expedite the planned functionality of the system.

1.2 Relation to Report 1

This report builds on the analysis and design presented in Report 1. However, where Report 1 focused on requirements, use cases, and system design, this report focuses on the implemented solution and its evaluation.

During the development some implementation details were adjusted to keep the scope manageable, and to fulfill the requirement. The final system still reflects the central ideas and requirements identified in the earlier report.

2 Implemented System Overview

This section briefly introduces the implemented system and its overall structure.

2.1 Implemented Functionality

The final system supports the following main features:

- creation of projects,
- creation and planning of activities,
- assignment of developers to activities,
- time registration on activities,
- absence registration,
- assignment of project leaders,
- project reporting,
- and progress tracking.

2.2 Architecture Overview

The implementation is structured into the following layers:

- **Domain layer:** core entities such as Project, Activity, Developer, Absence, and TimeRegistration,
- **Repository layer:** in-memory storage and lookup of system data,
- **Service layer:** application logic and coordination of operations,
- **User interface layer:** console and JavaFX-based interaction.

This layered design supports separation of concerns and improves testability.

2.3 Non-Implemented Features

2.4 Traceability to Report 1

3 Systematic Tests (White-Box Tests)

In this section, we present systematic white-box tests for selected methods. As required, we selected one method per group member. The chosen methods are action-oriented methods and not simple getters or setters.

3.1 Choice of Methods

The following methods were selected:

- `ProjectService.createProject(...)`
- `ActivityService.createActivity(...)`
- `ActivityService.addDeveloperToActivity(...)`
- `TimeRegistrationService.registerTime(...)`
- `AbsenceService.registerAbsence(...)`

These methods were chosen because they represent key system behaviour and contain validation logic, state changes, or coordination across multiple classes.

3.2 `ProjectService.createProject(...)`

3.2.1 Method

[Insert method code here]

3.2.2 Input Properties

Property	Description
Project name valid	Non-empty project name
Project name invalid	Empty or blank project name

Tabel 1: Input properties for `createProject(...)`

3.2.3 Concrete Test Cases

Test case	Input	Expected result
TC1	"WebShop"	Project is created successfully
TC2	" "	Exception: project name cannot be empty

Tabel 2: Concrete test cases for `createProject(...)`

3.3 ActivityService.createActivity(...)

3.2.4 Implementation of Tests

The planned test cases were implemented as executable tests in the project. These tests verify both valid and invalid input behaviour.

3.3 ActivityService.createActivity(...)

3.3.1 Method

[Insert method code here]

3.3.2 Input Properties

Property	Description
Project exists	The given project ID exists in the system
Project does not exist	The project ID is unknown
Activity name valid	Non-empty activity name
Activity name invalid	Empty activity name

Tabel 3: Input properties for `createActivity(...)`

3.3.3 Concrete Test Cases

Test case	Input	Expected result
TC1	Existing project, "Requirements"	Activity created
TC2	Unknown project, "Design"	Exception: project not found
TC3	Existing project, empty name	Exception: activity name cannot be empty

Tabel 4: Concrete test cases for `createActivity(...)`

3.4 ActivityService.addDeveloperToActivity(...)

3.4.1 Method

[Insert method code here]

3.5 TimeRegistrationService.registerTime(...)

3.4.2 Input Properties

Property	Description
Developer exists	The given developer initials exist
Developer does not exist	Unknown developer initials
Activity exists	The target activity exists in the chosen project

Tabel 5: Input properties for addDeveloperToActivity(...)

3.4.3 Concrete Test Cases

Test case	Input	Expected result
TC1	Existing developer assigned to existing activity	Developer added successfully
TC2	Unknown developer assigned to existing activity	Exception: developer not found

Tabel 6: Concrete test cases for addDeveloperToActivity(...)

3.5 TimeRegistrationService.registerTime(...)

3.5.1 Method

[Insert method code here]

3.5.2 Input Properties

Property	Description
Hours valid	Positive number and multiple of 0.5
Hours zero or negative	Invalid number of hours
Hours not multiple of 0.5	Invalid granularity
Activity exists	The activity exists in the specified project

Tabel 7: Input properties for registerTime(...)

3.5.3 Concrete Test Cases

Test case	Input	Expected result
TC1	2.5 hours	Time registration stored successfully
TC2	0.0 hours	Exception: invalid number of hours
TC3	1.3 hours	Exception: must be multiple of 0.5

Tabel 8: Concrete test cases for `registerTime(...)`

3.6 `AbsenceService.registerAbsence(...)`

3.6.1 Method

[Insert method code here]

3.6.2 Input Properties

Property	Description
Developer is free in period	Absence can be registered
Developer is busy in period	Absence must be rejected
Valid week interval	Start and end period form a meaningful interval

Tabel 9: Input properties for `registerAbsence(...)`

3.6.3 Concrete Test Cases

Test case	Input	Expected result
TC1	Vacation in free period	Absence registered successfully
TC2	Vacation in already booked period	Exception: vacation denied

Tabel 10: Concrete test cases for `registerAbsence(...)`

3.7 Summary of White-Box Testing

The systematic tests were derived from input partitions and boundary-relevant cases. The goal was to ensure that each selected method was tested for both normal behaviour and

3.7 Summary of White-Box Testing

relevant error conditions. These white-box tests complement the existing BDD tests and provide more explicit validation of implementation logic.

4 Design by Contract (Vedanta Sinha)

This section presents contracts for the same methods used in the systematic testing section. For each method, some preconditions are assumed, and postconditions are set. This is important as a logical system like code requires a systematic setup.

4.1 ProjectService.createProject(...)

4.1.1 Method with Assertions

```
1 public Project createProject(String name) { // creates and then
    stores the created project
2     if (name == null || name.isBlank()) {
3         throw new IllegalArgumentException("Project name cannot be
            empty");
4     }
5     String id = projectRepository.generateProjectId();
6     Project project = new Project(id, name);
7     projectRepository.add(project);
8     return project;
9 }
```

4.1.2 Contract

Precondition:

$$name \neq null \wedge name.trim().length() > 0$$

Postcondition:

$$result \neq null \wedge result.getName() = name$$

4.1.3 Discussion

The precondition ensures that only valid project names are accepted. The postcondition guarantees that a project object is created and stored consistently.

4.2 ActivityService.createActivity(...)

4.2.1 Method with Assertions

Listing 1: Activity service methods

```
1 public Activity createActivity(String projectId, String activityName)
    {
2     if (activityName == null || activityName.isBlank()) {
```

4.3 ActivityService.addDeveloperToActivity(...)

```
3         throw new IllegalArgumentException("Activity name cannot be empty");
4     }
5
6     Project project = projectRepository.findById(projectId)
7         .orElseThrow(() -> new IllegalArgumentException("Project not found: " + projectId));
8
9     Activity activity = new Activity(activityName);
10    project.addActivity(activity);
11    return activity;
12 }
```

4.2.2 Contract

Precondition:

$projectExists(projectId) \wedge activityName \neq null \wedge activityName.trim().length() > 0$

Postcondition:

$activity \in project.activities \wedge activity.getName() = activityName$

4.3 ActivityService.addDeveloperToActivity(...)

4.3.1 Method with Assertions

Listing 2: ActivityService constructor

```
1 }
2 public void addDeveloperToActivity(String projectId, String
3     activityName, String developerInitials) {
4     Activity activity = getActivity(projectId, activityName);
5
6     Developer developer = developerRepository.findByInitials(
7         developerInitials)
8         .orElseThrow(() -> new IllegalArgumentException("
9             Developer not found: " + developerInitials));
10
11     activity.addDeveloper(developer);
12 }
```

4.3.2 Contract

Precondition:

$projectExists(projectId) \wedge activityExists(projectId, activityName) \wedge developerExists(initials)$

4.4 TimeRegistrationService.registerTime(...)

Postcondition:

$developer \in activity.assignedDevelopers$

4.4 TimeRegistrationService.registerTime(...)

4.4.1 Method with Assertions

Listing 3: Register Time Method

```
1 public TimeRegistration registerTime(String developerInitials,
2                                     String projectId,
3                                     String activityName,
4                                     LocalDate date,
5                                     double hours) {
6
7     if (hours <= 0 || hours % 0.5 != 0) {
8         throw new IllegalArgumentException("Hours must be a positive
9         multiple of 0.5");
10    }
11
12    Developer developer = developerRepository.findByInitials(
13        developerInitials)
14        .orElseThrow(() -> new IllegalArgumentException(
15            "Developer not found: " + developerInitials));
16
17    Activity activity = projectRepository.findById(projectId)
18        .orElseThrow(() -> new IllegalArgumentException(
19            "Project not found: " + projectId))
20        .getActivities().stream()
21        .filter(a -> a.getName().equals(activityName))
22        .findFirst()
23        .orElseThrow(() -> new IllegalArgumentException(
24            "Activity not found: " + activityName));
25
26    TimeRegistration registration =
27        new TimeRegistration(developer, activity, date, hours);
28
29    activity.addTimeRegistration(registration);
30
31    return registration;
32 }
```

4.4.2 Contract

Precondition:

$hours > 0 \wedge hours \bmod 0.5 = 0$

Postcondition:

$$\exists r \in \text{activity.timeRegistrations} : r.\text{hours} = \text{hours}$$

4.5 AbsenceService.registerAbsence(...)

4.5.1 Method with Assertions

Listing 4: Register Absence Method

```

1 public Absence registerAbsence(String developerInitials,
2                               Absence.Type type,
3                               int startWeek, int startYear,
4                               int endWeek, int endYear) {
5
6     Developer developer = developerRepository.findByInitials(
7         developerInitials)
8         .orElseThrow(() -> new IllegalArgumentException(
9             "Developer not found: " + developerInitials));
10
11     if (type == Absence.Type.VACATION &&
12         isDeveloperBusyInPeriod(developer, startWeek, startYear,
13                                 endWeek, endYear)) {
14
15         throw new IllegalArgumentException(
16             "Vacation denied: developer is assigned to an
17             activity in this period");
18     }
19
20     Absence absence = new Absence(developer, type,
21                                   startWeek, startYear,
22                                   endWeek, endYear);
23
24     absenceRepository.add(absence);
25
26     return absence;
27 }

```

4.5.2 Contract

Precondition:

$$\text{developerExists}(\text{initials}) \wedge \text{periodIsValid}(\text{startWeek}, \text{startYear}, \text{endWeek}, \text{endYear})$$

Postcondition:

$$\text{absenceRegistered} \vee \text{exceptionThrown}$$

4.6 Discussion of Contracts

The contracts make assumptions and guarantees explicit. They improve readability, support debugging, and document the intended behaviour of the methods. In our implementation, assertions were added to enforce preconditions and postconditions directly in code.

5 Code Coverage (Viktor)

Code coverage is a quantitative metric that expresses the proportion of production code exercised at least once during a test suite execution. As Edsger W. Dijkstra famously noted, “testing can only reveal the presence of bugs, never their absence” coverage does not resolve this fundamental limitation, but it transforms an otherwise opaque question into a concrete, measurable property. A low coverage figure therefore guarantees that some code is untested, while a high figure is a necessary but not sufficient condition for confidence in the suite.

In the context of this project, the analysis serves two concrete purposes: to validate that the TDD workflow was genuinely followed in the service layer, and to document the concrete effect of adding the systematic white-box tests from Section 3.

5.1 Measurement Setup

Coverage was collected using Visual Studio Code’s built in "Run Tests with Coverage" action in the Test Explorer. VS Code’s Java Test Runner extension bundles the JaCoCo agent and attaches it to the JVM at test time, so no JaCoCo entry is required in `pom.xml`. JaCoCo performs on-the-fly bytecode instrumentation, probes are inserted when each class is loaded, recording which instructions and branches are executed. The results appear in VS Code’s `test coverage` panel.

The suite that underpins the measurement consists of 57 passing tests: 32 Cucumber BDD scenarios covering 12 use cases, and 25 JUnit white-box tests, 14 in `ProjectServiceTest`, 11 in `ActivityServiceTest` designed using equivalence partitioning and boundary value analysis.

5.2 Results

Table 11 summarises the per-package figures. The overall weighted total of 29% is dominated by the UI layer, which accounts for 880 of the 1245 executable lines and has 0% coverage by design (discussed in Section 5.3).

Package	Line coverage	Lines
<code>com.planner.domain</code>	79.77%	105
<code>com.planner.repository</code>	89.16%	53
<code>com.planner.service</code>	77.25%	207
<code>com.planner.ui</code>	0.00%	880
Total (weighted)	29.00%	1245

Tabel 11: Line coverage per package as reported by JaCoCo via VS Code’s `TEST COVERAGE` panel.

Figure 1 shows the top-level `test coverage` panel after running all 57 tests. The panel

is structured hierarchically from the `com.planner` root down through sub packages, only the package level is shown here, with the per-class views presented in Section 5.3.

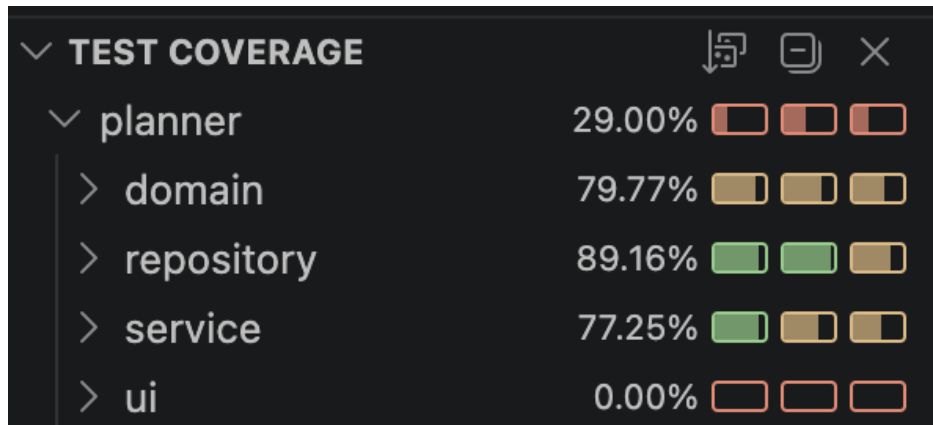


Figure 1: VS Code TEST COVERAGE panel at package level after executing all 57 tests.

5.3 Analysis by Layer

5.3.1 Repository layer 89.16%

Repositories are reached transitively on almost every scenario, giving the concrete implementations high coverage. The small gap below 100% is a cosmetic artefact: the three repository *interfaces* contain no executable bytecode, so JaCoCo counts them as 0%, which slightly lowers the package average. This is a mechanical consequence of applying the Dependency-Inversion principle, not a gap in test coverage.

5.3.2 UI layer 0%

The project brief permits the UI to be tested manually and states that the presentation layer does not need to be developed test-first. Because no automated test constructs a JavaFX `Scene`, the ~880 lines in `MainWindow.java` are never executed. Every user action in the UI is a direct delegation to a service method that is already covered; the 0% figure is therefore a deliberate design choice, not a testing gap.

5.3.3 Service layer 77.25%

This layer was developed strictly test first, so every public method in the six services has at least one Cucumber scenario. Figure 2 shows the per class breakdown. The 25 JUnit tests raise branch coverage further on `ProjectService` (85.25%) and `ActivityService` (82.18%) by targeting boundary values and edge cases, null/empty/whitespace names, week numbers 0, 1, 53, 54 that the BDD scenarios do not reach. `DeveloperService` sits notably lower at 23.08% because the project brief specifies that developers are imported from the HR file

rather than created through the system, leaving most of its methods unused in any scenario.

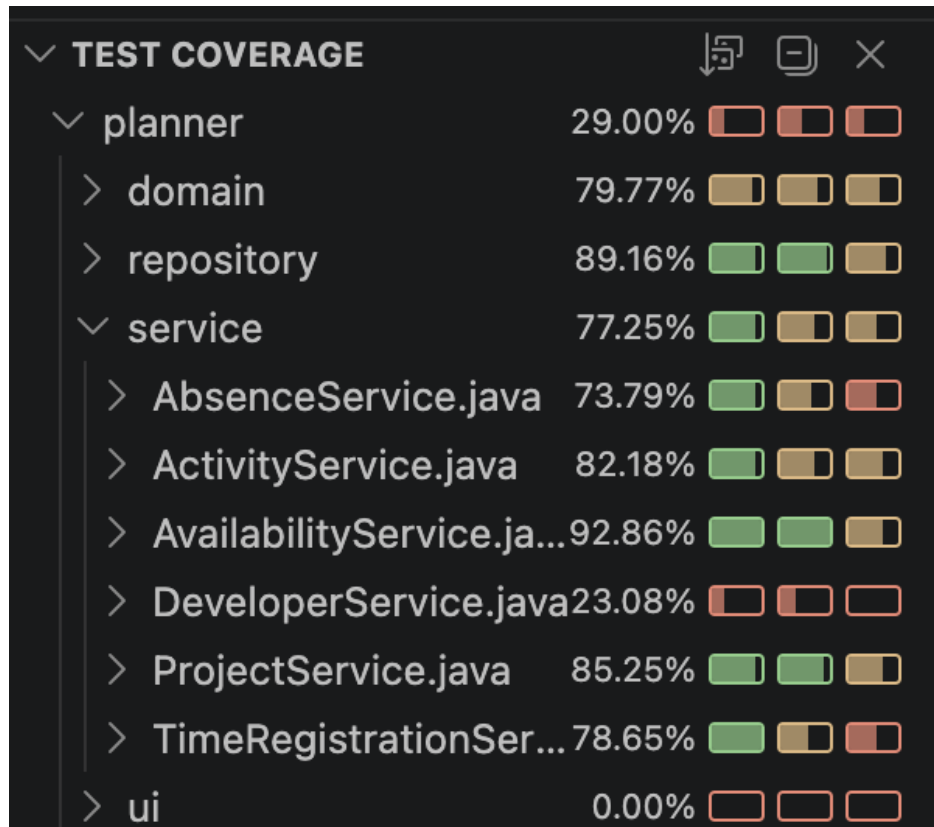
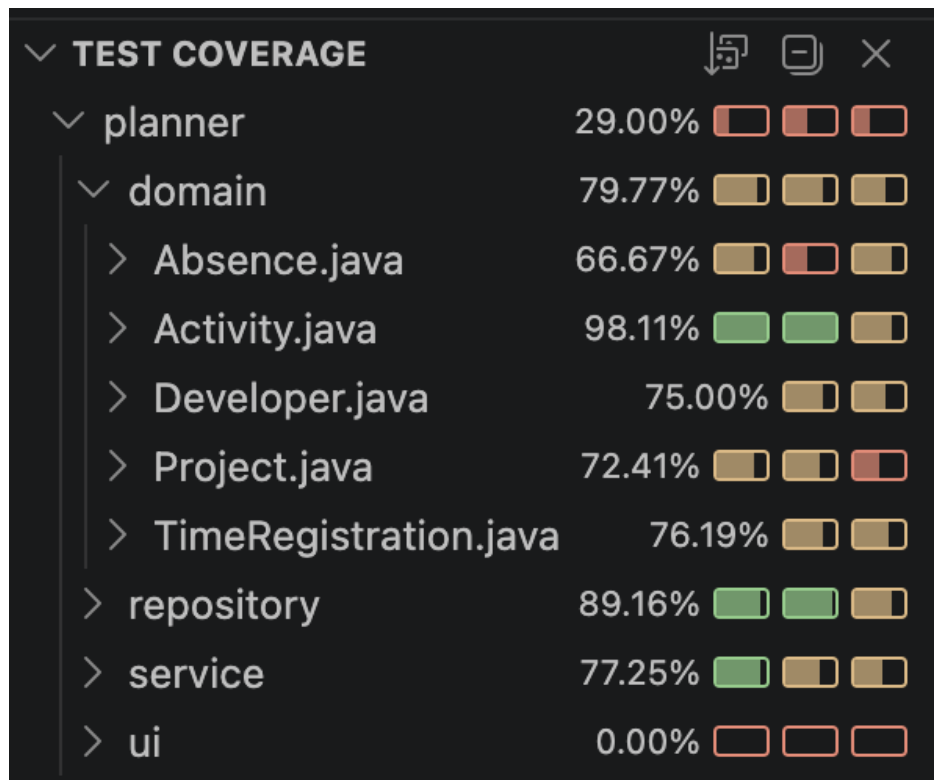


Figure 2: Per-class coverage within `com.planner.service`.

5.3.4 Domain layer 79.77%

Domain objects are thin data holders whose behaviour is exercised indirectly through the service tests. Figure 3 shows the per-class breakdown. `Activity.java` reaches 98.11% because activities are manipulated in almost every use case, while `Absence.java` stops at 66.67%, reflecting the smaller number of absence-related scenarios. The gap from 100% consists entirely of trivial getters (`toString`, `getType`, `setDate`) that no test calls directly; the project brief explicitly advises against testing bare getters.



Figur 3: Per-class coverage within `com.planner.domain`.

5.4 Effect of the White-Box Tests

The Cucumber scenarios verify business behaviour from the outside: they describe happy paths and the most prominent error cases in natural language, and they exercise whole call chains from the service layer down into the domain. Because they are written in terms of business outcomes, they are not designed to target individual branches or boundary values inside a method.

The 25 JUnit tests introduced in Section 3 fill exactly this gap. Using equivalence partitioning and boundary-value analysis, they target the code’s internal control-flow structure: the boundary values 0, 1, 53 and 54 in week-number validation; the three separate null/empty/whitespace cases for project names; and the conflict check between a new absence and an existing activity assignment. These are branches the Cucumber suite rarely or never reaches.

The combined suite produces a visibly tighter branch coverage on `ProjectService` and `ActivityService` than either approach achieves alone. This is not a coincidence; it is the empirical confirmation of the standard software-engineering claim that black-box specification-based testing and white-box structural testing are complementary methodologies, each reaching parts of the code the other leaves untouched.

5.5 Limitations

Coverage measures *execution*, not *correctness*. A test that calls a method without asserting anything contributes fully to coverage while verifying nothing. The figures in Table 11 should therefore be read alongside the Cucumber scenarios (behavioural correctness), the JUnit assertions (boundary correctness), and the design-by-contract checks in Section 4. Taken together, these four sources of evidence provide a considerably stronger argument for test adequacy than coverage alone.

5.6 Summary

Table 12 provides a compact overview of the main findings.

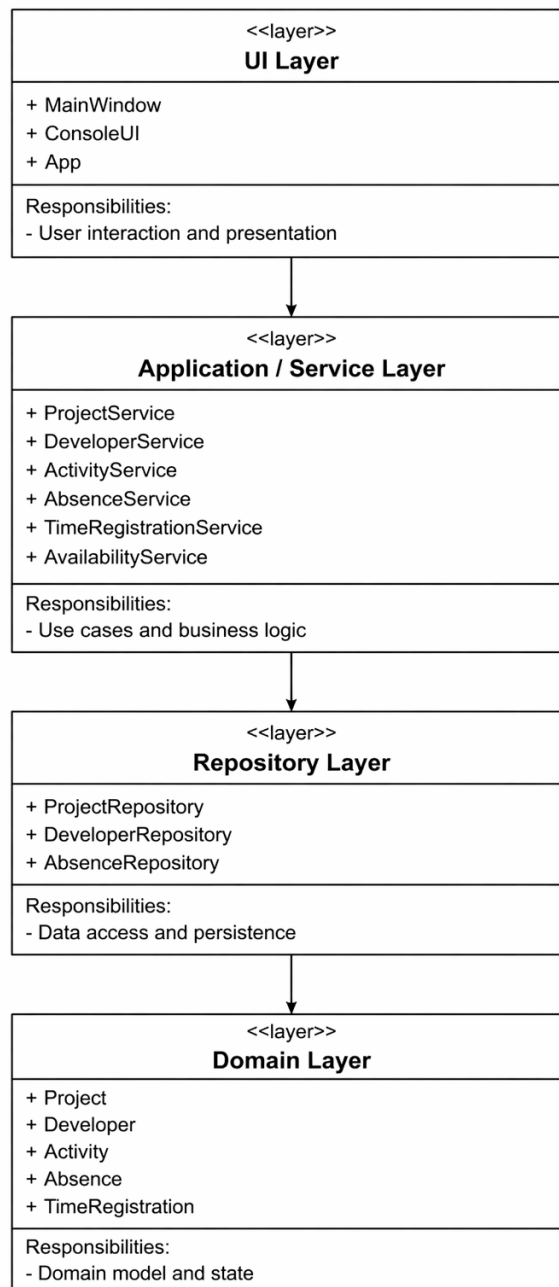
Layer	Coverage	Interpretation
Service	High (77.25%)	TDD followed; white-box tests close remaining branch gaps
Domain	Medium (79.77%)	Thin data classes; uncovered code is trivial getters only
Repository	High (89.16%)	Reached transitively; gap is interface bytecode artefact
UI	None (0.00%)	Deliberately excluded; business logic tested at service level

Tabel 12: Summary of coverage findings per architectural layer.

6 Design Patterns and SOLID Principles (nicolai)

6.1 Overall Design Choices

This project is centered around a layered architecture, where the code is separated into clear and separate *domain-classes*, *services*, *repositories*, and *user interface* as illustrated in 4



Figur 4: The layered achitecture of the system

This was done to create clear separation between files to make it possible for the group to work in parallel in tandem without too many dependencies between parts. Clarity, separation and function-independence has been in the groups design-scope from the start.

6.1.1 Domain Layer

The domain-layer carries the central objects of the program and includes the domain-files *Project*, *Activity*, *Developer*, *Absence* and *TimeRegistration*. These classes represent all data and relations within the system, and the program as a whole depends solely on them for the overall state of the system. User interaction, storage and higher application logic such as system rules, are kept out of this layer, to minimize dependencies to other parts of the system and avoid coupling between layers. For example, the *Project* class contains basic project information and a relation to its activities:

```

1      public class Project {
2          private final String id;
3          private final String name;
4          private Developer projectLeader;
5          private final List<Activity> activities = new ArrayList<>();
6
7          public Project(String id, String name) {
8              this.id = id;
9              this.name = name;
10         }
11     }

```

This way, the domain classes only store information about the given projects, and the relation to other domain objects, keeping presentation, general data storage and system rules out of the domain layer. These functions and responsibilities are handled by the service, repository and interface layers.

6.1.2 Repository Layer

The repository-layer is used to isolate general data storage from the rest of the system. The classes, *ProjectRepository*, *DeveloperRepository* and *AbsenceRepository* are in charge of saving and gathering data from the other layers in the system. In final iterations of the project, in-memory storage was introduced into these classes, keeping objects inside the repositories for easier testing. For an example, in the project repository

```

1      public class ProjectRepository {
2          private final List<Project> projects = new ArrayList<>();
3
4          public void add(Project project) {
5              projects.add(project);
6          }
7      }

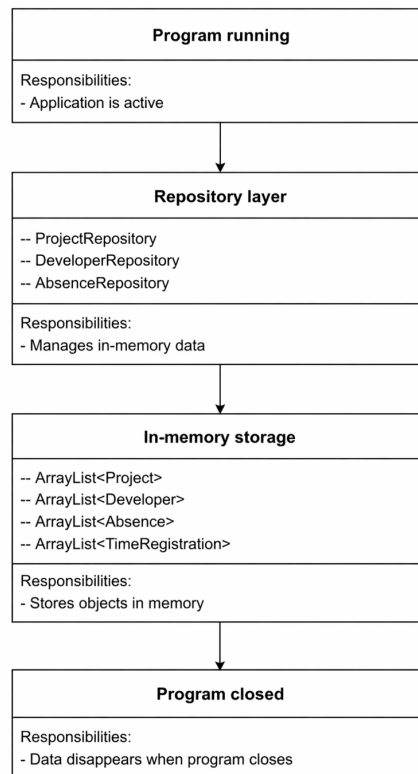
```

```

8      public Project findById(String id) {
9          // searches the list of projects
10     }
11 }

```

This way data is kept inside the single program as seen in 5



Figur 5: In-memory repositories

This was done as a deliberate design choice to support system testing as well as application logic for the group and further development, rather than having a persistent storage introducing unnecessary complexity for the required functionality of the program.

6.1.3 Service layer

While the state of the system is upheld and kept by the domain-layers, and data storage is kept by the repositories, the application logic is solely placed in the systems service layers. The layer, consisting of the classes *ProjectService*, *ActivityService*, *TimeRegistrationService*, and *AbsenceService*, each handle their own specific part of the systems functionality. This layer is responsible for coordinating all operations within the project, enforcing system rules, and keeping developers, leaders and projects aligned with eachother. For example, the creation of an activity is handled in the *ActivityService*:

6.2 SOLID-principles (dobbeltkonfekt skal rettes ti li final)

Listing 5: Activity creation in service layer

```
1 public void createActivity(String projectId, String activityName) {
2     Project project = projectRepository.findById(projectId);
3     // validation and logic
4     project.addActivity(new Activity(activityName));
5 }
```

As seen in the listing above, the *ActivityService* is end the *projectId* and *ActivityName*, then finds the correct project in the repository, validates and applies the necessary logic needed. The domain object itself does not handle how activities are created, and the repository is only responsible for storing and retrieving data.

6.1.4 User Interface Layer

The user interface layer is the interaction between the program and end user. This interface consist of a console-based interface and a graphical JavaFX-based interface. The UI layer is where all before mentioned logic is used or presented to the user. User input, and program functionality happens at this level, while keeping the actual logic hidden. Instead input is delegated to the before mentioned services, *ProjectService*, *ActivityService*, and *AbsenceService*. For an example, if a user wants to create a new project through the interface, the UI collects the input and call the relevant service-layer method needed. Validation, System logic and updates are then kept in these layers, leaving the interface layer as prestation and input only.

```
1 projectService.createProject(projectName);
```

skal rettes is muligvis This way, all system logic is kept from the end user, all while keeping all necessary information viewable in the *MainWindow*. It should be noted this is the overall politics of the interface, as there is quite an amount of UI-related code viewable instead of it being a full out developed GUI architecture. This was due to it being one of the last features developed, and was agreed upon as acceptable trade-off for focus on the project service and domain logic.

6.2 SOLID-principles (dobbeltkonfekt skal rettes ti li final)

In this project, focus has been on a structured system with clarity, testability, and further development in mind. To ensure this, the program is centered around a layered structure, with clear and separate *domain-classes*, *services*, *repositories*, and *user interface*. This was done to keep work separate and independent in a large group like ours. For the same reason, we have used the SOLID-principles to the extent we saw useful to a project this size and scope. Furthermore, we have not tried to conform our original ideas and code into other classic design patterns, if it did not solve any particular problem. Readability and understanding of the code produced, has had the priority to cater to different skill level, and further development in the process. The layered architecture, repository-patterns, and dependency injection are among the traditional design patterns, while SOLID-principles such as the SRP and DIP are among the principles most profound in our final system.

6.3 Single Responsible Principle

The program is highly structured around the Single Responsibility Principle through the before mentioned *domain-classes*, *services repositories*, and *UI-layers*. Every part of the system has a clear primary responsibility, which in contributes to a more clear and maintainable codebase.

As illustrated in figure 4, the separation between the layers is prominent, and each layer is responsible for a distinct part of the system, and the functionality are distributed accordingly.

6.3.1 Domain-classes

The layered structure and separation is easy to spot in the projects classes. Looking into the domain-classes, *Project*, *Activity*, *Developer*, *Absence* and *TimeRegistration*, we see that they primarily represents the relations and data within the system. Together, they keep system truth aligned and ensures the consistency of the data. They maintain the state that the program actually works within, making the project work.

For example, a *Project* contains a list of activities and basic project information:

```

1  public class Project {
2      private String name;
3      private Developer projectLeader;
4      private List<Activity> activities;
5  }
```

On the other hand, these classes are deliberately kept independent from the end user and data storage responsibilities. Data storage is instead handled by the repository layer.

6.3.2 Repositories

The repository layer is divided into classes such as *ProjectRepository*, *DeveloperRepository* and *AbsenceRepository*, each managing their own specific part of system-data. For an instance, the *projectRepository* is responsible for storing and retrieving the different projects inside the system:

```

1  public class ProjectRepository {
2      private List<Project> projects = new ArrayList<>();
3
4      public void add(Project project) {
5          projects.add(project);
6      }
7  }
```

```

8      public Project findById(String id) {
9          // lookup logic
10     }
11 }

```

Separation between the two layers, *domain and repositories*, ensures that changes to data storage does not directly affect the logic of the domain logic. This does not only make the system more manageable, but also flexible for further development if needed.

6.3.3 Service layer

While the state of the system is upheld and kept by the domain-layers, and data storage is kept by the repositories, the application logic is solely placed in the systems service layers. The layer, consisting of the classes *ProjectService*, *ActivityService*, *TimeRegistrationService*, and *AbsenceService*, each handle their own specific part of the systems functionality. This layer is responsible for coordinating all operations within the project, enforcing system rules, and keeping developers, leaders and projects aligned with eachother. For example, the creation of an activity is handled in the *ActivityService*:

Listing 6: Activity creation in service layer

```

1  public void createActivity(String projectId, String activityName) {
2      Project project = projectRepository.findById(projectId);
3      // validation and logic
4      project.addActivity(new Activity(activityName));
5  }

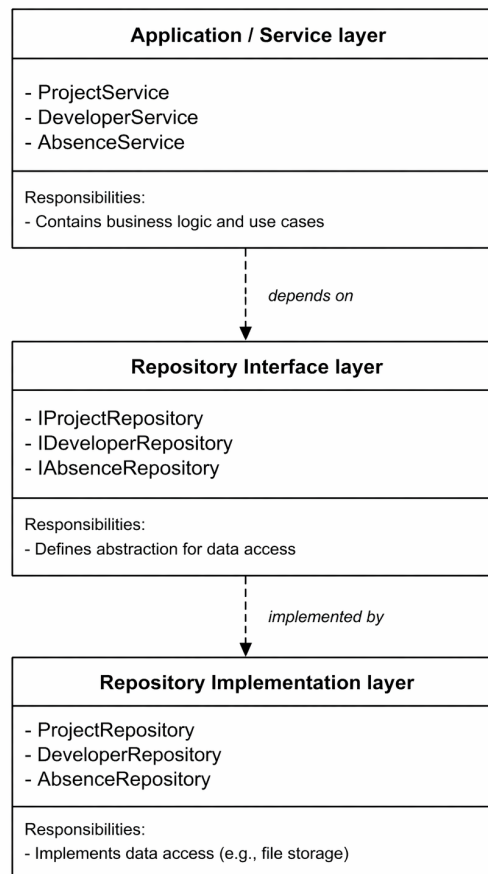
```

As seen in the listing above, the *ActivityService* is end the *projectId* and *ActivityName*, then finds the correct project in the repository, validates and applies the nessessary logic needed.

Through the layers just described, *Domain, Repository and Service*, it is clearly shown how the responsibilities are seperated, and a direct application of the SOLID-principle, Single responsibility Principle. Each layer well-defined, with a clear and limited respinsibility for the overall function of the system.

6.4 Dependency Inversion Principle

During late development of the final version of the project, the service layer was improved even further by applying repository interfaces such as *IProjectRepository*, *IAbsenseRepository* and *IDeveloperRepository*, using another SOLID principle, Dependency Inversion Principle. This was done to improve the overall design and future maintainability, with these introduced interfaces, service classes, *ProjectService*, *AbsenceService* and *DeveloperService*, no longer depend on the repository implementation, but use the *I-repositories* as abstractions. An example of this, can be seen in the constructor in the three serviceclasses, here *ProjectService*



Figur 6: DIP-flow (skal renames er træet)

```

1  public class ProjectService {
2      private final IProjectRepository projectRepository;
3
4      public ProjectService(IProjectRepository projectRepository) {
5          this.projectRepository = projectRepository;
6      }
7  }

```

This design choice, with DIP in mind, reduced the general dependency the service and storage had on each other. With this alteration, the general repository implementation happens outside the class, instead of being created internally. This makes the design more flexible in further development, because it makes it possible to replace general repository implementations without the need for change in the service logic itself.

6.5 Other SOLID-Principles

Single Responsibility and Dependency Inversion, are by design the most prominent and envisioned principles used in the Project. The remaining principles in SOLID, *Open/closed*, *Interface Segregation* and *Liskov* are only partially used in the design.

6.5.1 OCP, Open/Close principle

The Open/close principle, which is about the code being open for extension, but closed to modification, is only partially used in the service layer. Here, new functionality can be coded through service methods without actually modifying the domain classes directly.

```

1      public class ProjectService {
2
3      public void createProject(String name) {
4          // existing functionality
5      }
6
7      // New functionality
8      public void calculateProgress(String projectId) {
9          Project project = projectRepository.findById(projectId);
10         -> //new logic added without modifying Project class
11      }
12 }

```

This only points in the direction of real OCP, because of the need to change the service class to add functionalities to the code, and it can be concluded that the system is not designed around OCP extensibility.

The scope the group have had throughout the coding process has been on clarity, testability, and well structured understandable code that supports the structured implementation of the core functionality. To fully implement and support the open/close principle, would require a design overhaul, and result in complexity which we did not consider necessary for the overall purpose and functionality of the system we already build.

6.5.2 Interface Segregation Principle

The Interface Segregation states, that clients should not depend on functionality they do not need. Instead, the program should separate out the needed functionality in interfaces. In our program, ISP is not that prevalent. We did not feel the need to split the functionality up into separate interfaces more than it already is. Every client, the developer or project leader, have a very little interface already to begin with. The only place where ISP is somewhat used is again in the before mentioned repository interfaces, Separate out needed functionality in interfaces, *IProjectRepository*, *IAbsenceRepository* and *IDeveloperRepository*. However, this structure primarily follows the overall repository design, rather than being a direct result of applying ISP as a design principle.

6.5.3 Liskov Substitution Principle

At last the Liskov Substitution Principle, which states that subclasses should be changeable for their general base classes without affecting the functionality and correctness of the overall program. Or in simpler terms, classes should be able to inherit from other subclasses without change in the code. In our project, LSP is not used in the design. The system we made do not rely on inheritance of class hierarchies, but instead primarily focuses on composition and clear separation between the layers. As a result there are very few functions where substitution between subclasses are relevant, and the Liskov Substitution Principle are therefore not applied in the system intentionally. This reflects the overall design choice the group made to begin with, where composition and clarity was in a higher priority.

7 Conclusion

The final system implements the core functionality required for project planning, activity management, developer assignment, time registration, absence handling, and project tracking.

The implemented product is consistent with the main requirements selected in Report 1, although some parts were simplified to keep the scope manageable. In particular, the final implementation focuses on a coherent in-memory solution with automated tests and a working user interface.

Compared to the design in Report 1, the final implementation follows the same main structure, but some adjustments were made during implementation. These changes improved feasibility and reduced complexity while preserving the key system behaviour.

The testing work consists of both BDD-style feature tests and additional systematic white-box tests. Together, these provide stronger confidence in the correctness of the core application logic.

7.1 Reflection on the Development Process

The project was developed collaboratively across the group. Work was divided across analysis, implementation, testing, and documentation tasks. During the project, design decisions were refined as implementation details became clearer. This led to a final solution that is more concrete and implementation-oriented than the original design in Report 1.

7.2 Collaboration and Contribution Weights

Table 13 shows the contribution weights for each group member.

s240803	s234146	s245203	s245010	s240335
1.0	1.0	1.0	1.0	1.0

Tabel 13: Contribution weights for the group members

8 Contribution Table

Student Name	Student ID	Contribution Weight
[INSERT NAME]	[INSERT ID]	[INSERT VALUE] SERT NAME>SERT NAME
	[INSERT ID]	[INSERT VALUE] SERT NAME>SERT NAME
	[INSERT ID]	[INSERT VALUE] SERT NAME>SERT NAME
	[INSERT ID]	[INSERT VALUE]

Tabel 14: Contribution weights for the group members.

9 Appendix

9.1 Git Repository

The Git repository for the project is available at:

[Insert repository URL here]

9.2 Run Instructions

After unpacking the ZIP archive, the project can be imported as a Maven project in Eclipse, IntelliJ IDEA, or Visual Studio Code.

Run application

```
mvn javafx:run
```

Run tests

```
mvn test
```

The system includes at least one developer with the initials `huba`, as required in the project description.

Figurer

1	VS Code TEST COVERAGE panel at package level after executing all 57 tests. .	14
2	Per-class coverage within <code>com.planner.service</code>	15
3	Per-class coverage within <code>com.planner.domain</code>	16
4	The layered achitecture of the system	18
5	In-memory repositories	20
6	DIP-flow (skal renames er træet)	24

Tabeller

1	Input properties for <code>createProject(...)</code>	3
2	Concrete test cases for <code>createProject(...)</code>	3
3	Input properties for <code>createActivity(...)</code>	4
4	Concrete test cases for <code>createActivity(...)</code>	4
5	Input properties for <code>addDeveloperToActivity(...)</code>	5
6	Concrete test cases for <code>addDeveloperToActivity(...)</code>	5
7	Input properties for <code>registerTime(...)</code>	5
8	Concrete test cases for <code>registerTime(...)</code>	6
9	Input properties for <code>registerAbsence(...)</code>	6
10	Concrete test cases for <code>registerAbsence(...)</code>	6
11	Line coverage per package as reported by JaCoCo via VS Code's TEST COVERAGE panel.	13
12	Summary of coverage findings per architectural layer.	17
13	Contribution weights for the group members	27
14	Contribution weights for the group members.	28

Litteratur